MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 6 — Les Boucles

Automatiser la répétition — la puissance de l'itération

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

4 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 6 — Les Boucles **2**
0.1 La boucle while — Répéter tant que 2
0.2 La boucle for — Parcourir une séquence 3
0.3 La fonction range() — Générer des séquences 4
0.4 break — Sortir d'une boucle 5
0.5 continue — Passer à l'itération suivante 5
0.6 pass — Ne rien faire 6
0.7 La clause else des boucles 6
0.8 Les boucles imbriquées 7
0.9 enumerate() et zip() — Itérer avec style 8
0.10 Ce qui m'a le plus surpris dans ce chapitre 9
0.11 Résumé du Chapitre 6 9

Chapitre 6 — Les Boucles

À retenir

Les boucles sont l'un des concepts les plus puissants de la programmation. Elles permettent de **répéter automatiquement** un bloc de code — des dizaines, des milliers, des millions de fois si nécessaire. Sans boucles, on devrait réécrire chaque instruction à la main. Avec elles, une ligne de code peut faire le travail de mille.

La boucle while — Répéter tant que

La boucle `while` répète un bloc de code **tant qu'une condition est vraie**. Elle s'arrête dès que la condition devient fausse.

while — syntaxe de base

```python
# Syntaxe : while condition :
#                 bloc de code

compteur = 0

while compteur < 5:
    print(f"Compteur : {compteur}")
    compteur += 1    # IMPORTANT : toujours modifier la condition

#     Compteur : 0
#     Compteur : 1
#     Compteur : 2
#     Compteur : 3
#     Compteur : 4
```

Erreur fréquente

La **boucle infinie** est le piège classique du `while`. Si la condition ne devient jamais fausse, le programme tourne indéfiniment et se fige. Toujours s'assurer que la variable de la condition est **modifiée** à chaque tour de boucle. Pour arrêter une boucle infinie : `Ctrl + C` dans le terminal.

while — exemples pratiques

```python
# Demander un mot de passe jusqu'a ce qu'il soit correct
mot_de_passe = ""
tentatives = 0
```

```python
while mot_de_passe != "Python2024":
    mot_de_passe = input("Mot de passe : ")
    tentatives += 1
    if mot_de_passe != "Python2024":
        print(f"Incorrect. Tentative {tentatives}.")

print(f"Acces accorde apres {tentatives} tentative(s).")

# Compte rebours
decompte = 10
while decompte >= 0:
    print(decompte, end=" ")
    decompte -= 1
print("\n Decollage !")
#     10 9 8 7 6 5 4 3 2 1 0
#     Decollage !
```

La boucle for — Parcourir une séquence

La boucle `for` parcourt chaque élément d'une **séquence** (liste, chaîne, plage de nombres, etc.) et exécute le bloc pour chacun.

for — syntaxe de base

```python
# Parcourir une liste
fruits = ["pomme", "banane", "cerise", "mangue"]

for fruit in fruits:
    print(f"J'aime les {fruit}s !")

#     J'aime les pommes !
#     J'aime les bananes !
#     J'aime les cerises !
#     J'aime les mangues !

# Parcourir une chaine de caracteres
mot = "Python"
for lettre in mot:
    print(lettre, end="-")
#     P-y-t-h-o-n-
```

Ce qui m'a résisté

La variable de boucle (ici `fruit` ou `lettre`) est créée automatiquement par Python et prend la valeur de chaque élément à chaque tour. Au début, j'essayais de la déclarer avant la boucle — ce n'est pas nécessaire. Python la gère entièrement. Son nom est libre : on peut l'appeler comme on veut, mais par convention on choisit un nom **singulier** quand on parcourt une liste de **pluriels**.

La fonction range() — Générer des séquences

`range()` génère une séquence de nombres entiers. C'est la fonction la plus utilisée avec les boucles `for`.

range() — les 3 formes

```python
# range(stop)      de 0 a stop-1
for i in range(5):
    print(i, end=" ")
#     0 1 2 3 4

print()

# range(start, stop)      de start a stop-1
for i in range(2, 8):
    print(i, end=" ")
#     2 3 4 5 6 7

print()

# range(start, stop, step)      avec un pas
for i in range(0, 20, 4):
    print(i, end=" ")
#     0 4 8 12 16

print()

# Compter a rebours avec un pas negatif
for i in range(10, 0, -1):
    print(i, end=" ")
print("Boom !")
#     10 9 8 7 6 5 4 3 2 1 Boom !
```

range() — exemples concrets

```python
# Calculer la somme de 1 a 100
total = 0
for i in range(1, 101):
    total += i
print(f"Somme de 1 a 100 : {total}")    # 5050

# Table de multiplication
n = int(input("Table de multiplication de : "))
print(f"\n Table de {n} :")
for i in range(1, 11):
    print(f"  {n} x {i:2} = {n * i:3}")
```

À retenir

`range()` ne génère pas une liste — c'est un **objet range** qui produit les nombres à la demande. C'est très efficace en mémoire : `range(1000000)` n'occupe pas plus de mémoire que `range(5)`.

break — Sortir d'une boucle

`break` interrompt immédiatement la boucle en cours, quelle que soit la condition.

break — sortir d'une boucle

```python
# Chercher un element dans une liste
nombres = [4, 17, 3, 42, 8, 15, 99]
cible = 42

for nombre in nombres:
    if nombre == cible:
        print(f"Trouve : {cible} !")
        break   # Inutile de continuer
    print(f"Ce n'est pas {nombre}...")

#     Ce n'est pas 4...
#     Ce n'est pas 17...
#     Ce n'est pas 3...
#     Trouve : 42 !

# Securiser un nombre de tentatives
MAX_TENTATIVES = 3
for tentative in range(1, MAX_TENTATIVES + 1):
    reponse = input(f"Tentative {tentative}/3     Mot de passe
        : ")
    if reponse == "secret":
        print("Acces accorde !")
        break
    print("Incorrect.")
else:
    print("Compte bloque apr s 3 tentatives.")
```

continue — Passer à l'itération suivante

`continue` saute le reste du bloc de la boucle pour l'**itération courante** et passe directement à la suivante.

continue — sauter une itération

```python
# Afficher uniquement les nombres pairs
for i in range(1, 11):
    if i % 2 != 0:
```

```python
        continue    # Ignorer les impairs
    print(i, end=" ")
#     2 4 6 8 10

print()

# Filtrer des donnees invalides
notes = [15, -1, 18, 200, 12, 7, -5, 14]
notes_valides = []

for note in notes:
    if note < 0 or note > 20:
        print(f"Note invalide ignoree : {note}")
        continue
    notes_valides.append(note)

print(f"Notes valides : {notes_valides}")
moyenne = sum(notes_valides) / len(notes_valides)
print(f"Moyenne : {moyenne:.2f}")
```

pass — Ne rien faire

`pass` est une instruction qui ne fait **absolument rien**. Elle sert à créer un bloc vide syntaxiquement valide.

pass — placeholder

```python
# Boucle vide (utile pendant le developpement)
for i in range(10):
    pass    # A implementer plus tard

# Condition vide
age = 20
if age >= 18:
    pass    # Traitement a definir
else:
    print("Mineur.")

# Tres utilise pour les classes et fonctions vides
# (que l'on verra dans les chapitres suivants)
```

La clause else des boucles

En Python, les boucles peuvent avoir une clause `else` — une particularité unique que peu de langages possèdent.

else dans les boucles

```python
# Le bloc else s'execute quand la boucle se termine
# NORMALEMENT      c'est-a-dire sans break

# Exemple : chercher un nombre premier
nombre = 17
est_premier = True

for i in range(2, nombre):
    if nombre % i == 0:
        est_premier = False
        break
else:
    # Execute seulement si aucun break n'a eu lieu
    print(f"{nombre} est un nombre premier.")

# Meme logique avec while
compteur = 0
while compteur < 5:
    compteur += 1
else:
    print(f"Boucle terminee normalement. Compteur = {compteur}"
        )
```

Ce qui m'a résisté

Le `else` d'une boucle m'a complètement déstabilisé au début. En Python, `else` après une boucle ne signifie pas *"sinon la condition est fausse"*. Il signifie *"exécuté si la boucle s'est terminée sans* ***`break`****"*. C'est une sémantique totalement différente de celle des conditions. Il faut vraiment prendre le temps d'intégrer cette nuance — j'ai dû relire plusieurs fois avant de comprendre l'utilité réelle de cette construction.

Les boucles imbriquées

On peut placer une boucle à l'intérieur d'une autre pour traiter des **structures à deux dimensions**.

Boucles imbriquées

```python
# Table de multiplication complete
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i} x {j} = {i*j}", end="    ")
    print()    # Saut de ligne apres chaque ligne

#      1 x 1 = 1    1 x 2 = 2    1 x 3 = 3
#      2 x 1 = 2    2 x 2 = 4    2 x 3 = 6
#      3 x 1 = 3    3 x 2 = 6    3 x 3 = 9
```

```python

# Dessiner un rectangle d'etoiles
largeur = 6
hauteur = 4

for ligne in range(hauteur):
    for colonne in range(largeur):
        print("*", end=" ")
    print()

# Matrice      liste de listes
matrice = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

for ligne in matrice:
    for valeur in ligne:
        print(f"{valeur:3}", end="")
    print()
```

enumerate() et zip() — Itérer avec style

Python offre deux fonctions élégantes pour enrichir les boucles `for`.

enumerate() — index + valeur

```python
fruits = ["pomme", "banane", "cerise", "mangue"]

# Sans enumerate      maladroit
for i in range(len(fruits)):
    print(f"{i+1}. {fruits[i]}")

# Avec enumerate      elegant et pythonique
for index, fruit in enumerate(fruits, start=1):
    print(f"{index}. {fruit}")

#     1. pomme
#     2. banane
#     3. cerise
#     4. mangue
```

zip() — parcourir deux listes ensemble

```python
prenoms = ["Alice", "Bob", "Charlie"]
ages    = [25, 30, 22]
villes  = ["Paris", "Lyon", "Marseille"]
```

```python
for prenom, age, ville in zip(prenoms, ages, villes):
    print(f"{prenom}, {age} ans, habite a {ville}.")

#     Alice, 25 ans, habite a Paris.
#     Bob, 30 ans, habite a Lyon.
#     Charlie, 22 ans, habite a Marseille.
```

Ce qui m'a résisté

`enumerate()` et `zip()` m'ont révélé ce que signifie vraiment **écrire du Python**. Avant de les connaître, j'utilisais `range(len(liste))` partout — ce qui fonctionne, mais c'est considéré comme non-pythonique. Python encourage à penser en termes d'**itération directe**, pas d'index. Ce changement de paradigme prend du temps, mais il transforme radicalement la qualité du code.

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. Le else d'une boucle — une logique contre-intuitive. Dans tous les langages que j'avais effleurés, `else` suit toujours un `if`. En Python, il peut suivre une boucle — et sa signification change complètement. Il s'exécute quand la boucle se termine **sans interruption**. Comprendre cette nuance m'a demandé plusieurs exemples concrets avant que ça devienne naturel.

2. range() ne crée pas de liste. J'avais imaginé que `range(1000000)` créait un million de nombres en mémoire. En réalité, c'est un **générateur paresseux** — il calcule chaque nombre à la demande. Cette distinction entre ce qu'on voit et ce qui se passe réellement en mémoire m'a ouvert les yeux sur l'importance de l'efficacité en Python.

3. enumerate() et zip() changent tout. Découvrir ces deux fonctions m'a fait réaliser qu'il existe souvent une façon *"pythonique"* d'écrire les choses — plus courte, plus lisible, plus élégante. Python ne se contente pas d'être fonctionnel. Il invite à **bien** écrire le code. C'est une philosophie autant qu'un langage.

Résumé du Chapitre 6

Ce que j'ai appris dans ce chapitre :

- `while` — répète tant qu'une condition est vraie. Toujours modifier la condition pour éviter la boucle infinie.
- `for` — parcourt chaque élément d'une séquence. Simple, lisible, puissant.

- `range(start, stop, step)` — génère des séquences de nombres à la demande.
- `break` — interrompt la boucle immédiatement.
- `continue` — saute l'itération courante et passe à la suivante.
- `pass` — bloc vide syntaxiquement valide, utile en développement.
- `else` après une boucle — s'exécute uniquement si aucun `break` n'a eu lieu.
- `enumerate()` et `zip()` — itérer avec index et plusieurs listes simultanément.

"La puissance de l'ordinateur, c'est la répétition sans fatigue. La boucle, c'est là où la machine surpasse l'humain."

KETOTSA AMÉVI CLAUDE — Mon apprentissage Python | Chapitre 6 sur 34